

AZAAN INTERNATIONAL SCHOOL
CLASS XII | COMPUTER SCIENCE |

=====

Evaluating Arithmetic Operations: An **arithmetic expression** consists of **operators** and **operands**. Python evaluates an arithmetic expression by following the **operator precedence** (priority of operators) and **associativity** rules.

Steps to Evaluate an Arithmetic Expression

Python evaluates an arithmetic expression in the following order:

1. **Checks the operator precedence.**
2. **Evaluates the operands (values).**
3. **Performs implicit type conversion (if required).**
4. **Executes the operation.**
5. **Replaces the sub-expression with the result.**
6. **Repeats the process until the final result is obtained.**

Type Promotion (Implicit Type Conversion): When different numeric data types are used in an expression, Python automatically converts the smaller data type into the larger data type.

Example 1: `print(10 + 2.5)`

Output: 12.5

Operator Precedence (Highest to Lowest)

Operator	Description
()	Parentheses (Grouping)
**	Exponentiation
~X	Bitwise nor
+X, -X	Unary plus, Unary minus
*, /, //, %	Multiplication, Division, Floor Division, Modulus
+, -	Addition, Subtraction
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR

Operator	Description
<, <=, >, >=, ==, !=, is, is not	Relational & Identity Operators
Not x	Boolean NOT
and	Boolean AND
or	Boolean OR

Consider below given expressions what. What will be the final result and final data type

(a) a, b= 3,6 then find : c=b/a

Ans. Here, a=3 & b=6 so , c=b/a=6/3= 2.0

1.8.2 Evaluating Relational Expressions: A relational expression

compares two values using **relational operators**. The result of every relational expression is always a **Boolean value** (True or False).

Chained Comparisons: Python allows multiple comparisons in a single expression.

Example 1: $p > q < y$

Internally interpreted as: $(p > q) \text{ and } (q < y)$

Example 2: $a \leq N \leq b$

Internally interpreted as: $(a \leq N) \text{ and } (N \leq b)$

1.8.3 Evaluating Logical Expressions: A logical expression combines one or more conditions using **logical operators**. The result is always **True** or **False**.

Rules for Evaluating Logical Expressions

Rule 1: Arithmetic expressions are evaluated **before** logical operators.

Rule 2: The precedence of logical operators is: (i) not (ii) and (iii) or

Rule 3 (Short-Circuit Evaluation)

or operator : If the first condition is True, Python does not check the second condition because the result will already be True.

&and operator : If the first condition is False, Python does not check the second condition because the result will already be False.

Example: $(5 < 10) \text{ and } (10 < 5) \text{ or } (3 < 18) \text{ and not } (8 < 18)$

Output: False

1.8.4 Type Casting (Explicit Type Conversion): *Type Casting is the process of converting one data type into another **manually** by the programmer. It is also called **Explicit Type Conversion**.*

Syntax: `datatype(expression)`

Example 1: `a = 3`

`b = 5.0`

`print(int(b))`

Output: 5

Example 2: `x = "25"`

`print(int(x))`

Output: 25

1.8.5 MATH LIBRARY FUNCTIONS

Python's standard library provides a module namely **math** for math related functions that work with all number types except for complex numbers.

In order to work with functions of math module, you need to first import it to your program as “import math”

Then you can use math library's functions as: `math.<function-name>`

Function	Description	Example
ceil()	Returns the smallest integer not less than num .	<code>math.ceil(1.03) → 2</code>
sqrt()	Returns the square root of num . If num < 0 , a domain error occurs.	<code>math.sqrt(81.0) → 9.0</code>
exp()	Returns e raised to the power arg .	<code>math.exp(2.0) → e²</code>
fabs()	Returns the absolute value of num .	<code>math.fabs(1.0)</code>

Function	Description	Example
floor()	Returns the largest integer not greater than num .	<code>math.floor(1.03) → 1</code>
log()	Returns the natural logarithm of num .	<code>math.log(1.0)</code>
log10()	Returns the base-10 logarithm of num .	<code>math.log10(1.0)</code>
pow()	Returns base raised to the power exp .	<code>math.pow(3, 2) → 9</code>
sin()	Returns the sine of arg (in radians).	<code>math.sin(30)</code>
cos()	Returns the cosine of arg (in radians).	<code>math.cos(30)</code>
tan()	Returns the tangent of arg (in radians).	<code>math.tan(30)</code>
degrees()	Converts angle x from radians to degrees.	<code>math.degrees(30)</code>
radians()	Converts angle x from degrees to radians.	<code>math.radians(30)</code>
fmod()	Returns the modulus of x/y with higher precision for floats.	<code>math.fmod(20, 6) → 2.0</code>
factorial	Returns the factorial of arg as an integer.	<code>math.factorial(4) → 24</code>
gcd	Returns the greatest common divisor of integers x and y .	<code>math.gcd(8, 6) → 2,</code>

1.9 STATEMENT FLOW CONTROL: In a program, statements may be executed: **Sequentially, Selectively, Iteratively**

- A **conditional** is a statement set which is executed, on the basis of result of a condition. A **loop** is a statement set which is executed repeatedly, until the end condition is satisfied.
- Following are the types of statements.

1. Compound Statement: A compound statement represents a group of statements executed as a unit.

Syntax:

<compound statement header> :

 <indented body containing multiple simple and/or compound statements>

Example: for i in range(5):

 print(i)

2. Simple Statement: Any single executable statement is a simple statement in Python.

3. Empty Statement: The simplest statement is the empty statement i.e., a statement which does nothing. In Python an empty statement is the **pass** statement.

The If Conditionals : The **if conditional statement** is used to make decisions in a program. It checks a condition. If the condition is **True**, the statements inside the if block are executed. If the condition is **False**, those statements are not executed.

Python provides three types of conditional statements:

(i) Plain if Conditional Statement: The **if statement** is the simplest decision-making statement in Python. It checks a condition. If the condition is **True**, the statements inside the if block are executed. If the condition is **False**, the program skips the block and continues with the next statement.

Syntax:

if condition:

 statement1

 statement2....

Example

`a=10`

`b=20`

`if a>b:`

`print(a)`

(ii) The if-else Conditional Statement: The **if-else statement** is used when a program has two possible outcomes. If the condition is **True**, the statements inside the if block are executed. Otherwise, the statements inside the else block are executed.

Syntax:

`if condition:`

`statement1`

`statement2`

`else:`

`statement3`

`statement4`

Example

`a=10`

`b=20`

`if a>b:`

`print(a)`

`else:`

`print(b)`

(iii) if-elif Conditional Statement: The **if-elif statement** is used to check **multiple conditions** one after another. If the first condition is **False**, Python checks the next condition (elif). As soon as one condition becomes **True**, its corresponding block is executed and the remaining conditions are skipped. When no condition is **True**, the **else** block (if present) is executed.

Syntax of if-elif Statement

`if condition1:`

`statement`

elif condition2:

statement

else:

statement

Example:

a = 10

b = 20

if a > b:

print(a)

elif a < b:

print(b)

else:

print("Both are equal")

(iv) Nested if Statements: A **Nested if statement** is an if statement placed **inside another if, elif, or else block**. It is used when a program needs to check **more than one condition**.

Syntax

if condition1:

if condition2:

statement

else:

statement

else:

statement

Example

a = 10

b = 20

if a < b:

if a > 5:

print("a is smaller than b and greater than 5")

Output

a is smaller than b and greater than 5

Storing Conditions: **Storing Conditions** means assigning the result of a condition (**True** or **False**) to a variable and then using that variable in if statements.

This makes the program **more readable** and **reduces repetition**.

Syntax

```
condition_name = condition
```

```
if condition_name:
```

```
    statement
```

Example

```
a = 1
```

```
b = 2
```

```
c = 3
```

```
all_conditions = (a == 1 and b == 2 and c == 3)
```

```
if all_conditions:
```

```
    print("Condition fulfilled")
```

Output: Condition fulfilled

1.11 Looping Statements: A **loop** is used to execute a block of code repeatedly until a condition is satisfied or a sequence is completed.

Python provides two types of loops:

- **for loop** (used when the number of iterations is known)
- **while loop** (used when the number of iterations is not known)

1.11.1 The for Loop: The **for loop** is used to execute a block of code **for every element in a sequence** such as a list, string, tuple, or range.

Syntax

```
for variable in sequence:
```

```
    statement
```

Example

```
for i in [10, 15, 20, 25]:
```

```
    print(i + 2)
```

Output

```
12
```

```
17
```

```
22
```

```
27
```

The range() Function: The **range()** function generates a sequence of integers and is commonly used with the for loop.

Three ways to define range function:

range(stop)

range(start, stop)

range(start, stop, step)

Examples

Function	Output
<i>range(5)</i>	0, 1, 2, 3, 4
<i>range(2,6)</i>	2, 3, 4, 5
<i>range(2,10,2)</i>	2, 4, 6, 8
<i>range(10,5,-1)</i>	10, 9, 8, 7, 6

1.11.2 The while Loop: The **while loop** executes a block of code **as long as the given condition is True**. It is called an **entry-controlled loop** because the condition is checked before entering the loop.

Syntax

while condition:

statement

Example

i = 1

while i <= 3:

print(i)

i += 1

Output

1

2

3

Difference Between for and while Loop:

for Loop	while Loop
<i>Used when the number of iterations is known.</i>	<i>Used when the number of iterations is unknown.</i>
<i>Iterates over a sequence.</i>	<i>Executes until the condition becomes False.</i>

1.12 Jump Statements

Python provides two jump statements:

- **break**
- **continue**

(i) The break Statement: The **break** statement immediately terminates the loop and transfers control to the first statement after the loop.

Syntax

`break`

Example

```
for i in range(1,6):
```

```
    if i == 3:
```

```
        break
```

```
    print(i)
```

Output

1

2

The continue Statement: The **continue** statement skips the remaining statements of the current iteration and starts the next iteration of the loop.

Syntax

`continue`

Example

```
for i in range(1,6):
```

```
    if i == 3:
```

```
        continue
```

```
    print(i)
```

Output

1

2

4

5

Difference Between break and continue

break	continue
Terminates the loop.	Skips the current iteration only.
Control moves outside the loop.	Control moves to the next iteration.

1.13 More on Loops

1.13.1 Loop else Statement: Python loops can have an optional **else** block. The **else** block executes **only when the loop finishes normally**. It **does not execute** if the loop is terminated using a **break** statement.

Syntax

```
for variable in sequence:
    statements
else:
    statements
```

Example

```
for i in range(1,4):
    print(i)
else:
    print("Loop completed")
```

Output

```
1
2
3
Loop completed
```

1.13.2 Nested Loops: A **Nested Loop** is a loop inside another loop. The **inner loop** executes completely for every iteration of the **outer loop**.

Syntax

```
for i in range(...):
    for j in range(...):
        statement
```

Example:

```
for i in range(1, 3):
    for j in range(1, 3):
        print("*")
```

Output:

```
*
*
*
*
```

BASIC PROGRAMS

1 Write a program to input a number and print its cube.

```
num = float(input('Enter a number: '))
num_cube = num * num * num
print('The cube of', num, 'is', num_cube)
```

OUTPUT:

```
Enter a number: 2
The cube of 2.0 is 8.0
```

1.2 Write a program to input a number and print its square root.

```
num = float(input('Enter a number: '))
num_sqrt = num ** 0.5
print('The square root of', num, 'is', num_sqrt)
```

output:

```
Enter a number: 2
The square root of 2.0 is 1.4142135623730951
```

1.3 Write a program that inputs an integer in range 0 - 999 and then prints if the integer entered is a 1/2/3 digit number.

```
num = int(input("Enter a number (0..999) : "))
if num < 0:
    print("Invalid entry. Valid range is 0 to 999.")
elif num < 10:
    print("Single digit number is entered")
elif num < 100:
    print("Two digit number is entered")
elif num <= 999:
    print("Three digit number is entered")
else:
    print("Invalid entry. Valid range is 0 to 999.")
```

Sample runs of this program are given below :

```
Enter a number (0..999) : -3
Invalid entry. Valid range is 0 to 999.
```

```
Enter a number (0..999) : 4
Single digit number is entered
```

Enter a number (0..999) : 100

Three digit number is entered

Enter a number (0..999) : 10

Two digit number is entered

Enter a number (0..999) : 3000

Invalid entry. Valid range is 0 to 999.

1.4 Write a program that inputs an integer in range 0 - 999 and then prints if the integer entered is a 1/2/3 digit number. Use Nested if statements.

```
num = int(input("Enter a number (0..999) : "))
if num < 0 or num > 999:
    print("Invalid entry. Valid range is 0 to 999.")
else:
    if num < 10:
        print("Single digit number is entered")
    else:
        if num < 100:
            print("Two digit number is entered")
        else:
            print("Three digit number is entered")
```

OUTPUT:

Enter a number (0..999) : 3

Single digit number is entered

1.5 Write a program to print cubes of numbers in the range 15 to 20.

for i in range(15, 21):

print("Cube of number", i, end = ' ')

*print("is", i ** 3)*

output:

Cube of number 15 is 3375

Cube of number 16 is 4096

Cube of number 17 is 4913

Cube of number 18 is 5832

Cube of number 19 is 6859

Cube of number 20 is 8000

1.7 Write a program to print square root of every alternate number in the range 1 to 10.

```
for i in range(1, 10, 2):  
    print("Square root of", i, "is", (i ** 0.5))
```

1.8 Write a program that multiplies two integer numbers without using the * operator, using repeated addition.

main.py



Share

Run

```
1 n1 = int(input("Enter first number: "))  
2 n2 = int(input("Enter second number: "))  
3 product = 0  
4 count = n1  
5 while count > 0:  
6     count = count - 1  
7     product = product + n2  
8 print("The product of", n1, "and", n2, "is", product)
```

AZAAN INTERNATIONAL SCHOOL
CLASS XII | COMPUTER SCIENCE |

=====

Strings in Python: A String is a sequence of characters enclosed within single (' '), double (" "), or triple quotes ("' '").

Example: name = "Python"

String Indexing: Each character in a string has an index.

- Forward Indexing: Starts from 0
- Backward Indexing: Starts from -1

Example: a= "PYTHON"

Character	P	Y	T	H	O	N
Forward Index	0	1	2	3	4	5
Backward Index	-6	-5	-4	-3	-2	-1

Accessing Characters: Characters of string can be accessed with the help of index position. **Syntax:** <string_name>[index]

Example: name[0] # P

 name[-1] # N

Item Assignment Not Supported: Strings are immutable, which means individual characters cannot be changed.

Example

name = "hello"

name[0] = "P"

Output: TypeError

Traversing a String: Traversing means accessing each character of a string one by one.

Example: word = "Python"

for ch in word:

 print(ch)

Output

P
y
t
h
o
n

String Operators:

1. Concatenation Operator (+): The + operator joins two strings and makes it as one combined string.

Example: `print("Hello" + " World")`

Output: Hello World

Note: Both operands must be strings.

2. Replication Operator (*): The * operator repeats a string multiple times.

Example: `print("Hi " * 3)`

Output: Hi Hi Hi

3. Membership Operators: Two membership operators are: in, not in

(i) in: Returns True if the character or substring exists.

Example: `"Py" in "Python"`

Output: True

(ii) not in: Returns True if the character or substring does not exist.

Example: `"Java" not in "Python"`

Output: True

4. Comparison Operators: Strings can be compared using: <, >, <=, >=, ==, !=. Comparison is based on ASCII/Unicode values (Dictionary Order).

Example: `"ABC" == "abc"`

Output: False

ord() Function: The ord() function returns the ASCII/Unicode value of a character.

Syntax: `ord(character)`

Example: `ord('A')`

Output: 65

chr() Function: The chr() function returns the character for the given ASCII value.

Syntax: chr(number)

Example: chr(65)

Output: A

String Slicing: String slicing is used to extract a part of a string.

Syntax: string[start : stop]

Example: word = "Python"
print(word[1:4])

Output: yt

Common String Functions: Python offers many built-in functions and methods for string manipulation.

Syntax: <stringObject>.<method name>()

capitalize(): Converts the first letter into uppercase.

Example: "python".capitalize()

Output: Python

find(sub[,start[,end]]): Returns the index of the first occurrence of a substring. Returns -1 if not found.

Example: "Python".find("th")

Output: 2

isalnum(): Returns True if the string contains only letters and digits.

Example: "abc123".isalnum()

Output: True

isalpha(): Returns True if all characters are alphabets.

Example: "Python".isalpha()

Output: True

isdigit(): Returns True if all characters are digits.

Example: "12345".isdigit()

Output: True

isspace(): Returns True if the string contains only spaces.

Example: " ".isspace()

Output: True

islower(): Checks whether all letters are lowercase.

Example: "python".islower()

Output: True

isupper(): Checks whether all letters are uppercase.

Example: "PYTHON".isupper()

Output: True

istitle(): Checks whether every word starts with a capital letter.

Example: "My Name".istitle()

Output: True

lower(): Converts all letters into lowercase.

Example: "PYTHON".lower()

Output: python

upper(): Converts all letters into uppercase.

Example: "python".upper()

Output: PYTHON

title(): Converts the first letter of every word into uppercase.

Example: "my name is ram".title()

Output: My Name Is Ram

startswith(): Checks whether the string starts with a given value.

Example: "Python".startswith("Py")

Output: True

endswith(): Checks whether the string ends with a given value.

Example: "Python".endswith("on")

Output: True

swapcase(): Converts uppercase letters to lowercase and lowercase letters to uppercase.

Example: "PyThOn".swapcase()

Output: pYtHoN

partition(): Splits a string into three parts.

Example: "Python is Easy".partition("is")

Output: ('Python ', 'is', ' Easy')

count(): Returns the number of occurrences of a substring.

Example: "banana".count("a")

Output: 3

lstrip([chars]): Removes characters from the **beginning (left side)** of a string.

Example: "hello".lstrip("he")

Output: 'llo'

rstrip([chars]): Removes characters from the **end (right side)** of a string.

Example: "hello".rstrip("lo")

Output : 'hel'

2.3 LISTS IN PYTHON: A List is a standard data type in Python used to store a collection of values in a single variable.

- A list is enclosed within square brackets [].
- Elements are separated by commas (,).
- A list can store same or different data types & allows duplicate values.
- Lists are mutable, which means their elements can be changed after creation.

Syntax: ListName = [value1, value2, value3]

Examples: L1 = [1, 2, 3], L2 = ['a', 'b', 'c'], L3 = [10, 'Python', 3.5], L4 = []

2.3.1 Creating Lists: A list can be created in different ways.

(i) Creating an Empty List: An empty list contains no elements. We have two methods to create a list

1. L = []

2. L = list()

(ii) Creating a List with Values: Store elements inside square brackets.

Example:

1. marks = [85, 90, 95]

2. fruits = ["Apple", "Mango", "Orange"]

(iii). Creating a List from an Existing Sequence: The list() function converts another sequence into a list. The sequence can be String , Tuple, Another List

Syntax: list(sequence)

Example 1: From String

```
L = list("HELLO")
```

```
print(L)
```

Output: ['H', 'E', 'L', 'L', 'O']

(iv). Creating a List from Keyboard Input

Method 1

```
L = list(input("Enter elements: "))
```

Example

Input

12345

Output

['1', '2', '3', '4', '5']

Method 2

To enter a complete list:

```
L = eval(input("Enter list: "))
```

Example

Input: [10,20,30]

Output:[10, 20, 30]

2.3.2 Lists vs Strings

Lists and Strings have many similarities, but they are also different.

(A) Similarities between Lists and Strings

Similar Feature	Syntax / Operator	List Example	String Example
Length	<code>len(seq)</code>	<code>len([10,20,30]) → 3</code>	<code>len("ABC") → 3</code>
Slicing	<code>seq[start : stop]</code>	<code>[10,20,30,40][1:3] → [20,30]</code>	<code>"HELLO"[1:4] → "ELL"</code>
Membership	<i>in and not in</i>	<code>20 in [10,20] → True</code>	<code>'A' in "AB" → True</code>
Concatenation	<code>+</code>	<code>[1,2] + [3,4] → [1,2,3,4]</code>	<code>"AB" + "CD" → "ABCD"</code>
Replication	<code>*</code>	<code>[1,2] * 2 → [1,2,1,2]</code>	<code>"AB" * 2 → "ABAB"</code>
Element Access	<code>seq[index]</code>	<code>['a','b','c'][2] → 'c'</code>	<code>"abc"[2] → 'c'</code>

B. Difference between Lists and Strings

Lists	Strings
<i>Mutable</i>	<i>Immutable</i>
<i>Elements can be modified</i>	<i>Characters cannot be modified</i>
<i>Stores references to objects</i>	<i>Stores characters</i>

Example: L = ['a','b','c']

L[0]='A'

print(L)

Output

['A','b','c']

This is possible because lists are mutable.

2.3.3 List Operations: *List operations allow us to work with list elements easily.*

1. Traversing a List: *Traversing means accessing each element of a list one by one.*

Syntax: for variable in List:

statement

Example: L = ['P','Y','T','H','O','N']

for x in L:

print(x)

Output

P

Y

T

H

O

N

2. Joining Lists (Concatenation): *The + operator joins two lists.*

Example: L1 = [1,4,9], L2 = [6,12,20]

print(L1 + L2)

Output

[1,4,9,6,12,20]

Note: Both operands must be lists.

3. Repeating (Replicating) Lists: The * operator repeats the list.

Example: L = [1,2,3]

```
print(L*3)
```

Output: [1,2,3,1,2,3,1,2,3]

4. Slicing Lists: Slicing extracts a portion of the list.

Syntax: List[start:stop]

Example: L = [10,12,14,20,22,24,30]

```
print(L[2:5])
```

Output: [14,20,22]

Slicing with Step Value

Syntax: List[start:stop:step]

Example: L = [10,12,14,20,22,24,30,32,34]

```
print(L[0:9:2])
```

Output

[10,14,22,30,34]

2.3.4 List Manipulation: List Manipulation means performing different operations on a list such as adding, updating, deleting, or modifying its elements.

The common list manipulation operations are:

- Appending Elements
- Updating Elements
- Deleting Elements

1. Appending Elements to a List: The append() method is used to add one element at the end of a list.

Syntax: ListName.append(item)

Example: fruits = ["Apple", "Mango"]

```
fruits.append("Orange")
```

```
print(fruits)
```

Output

['Apple', 'Mango', 'Orange']

2. Updating Elements in a List: Lists are mutable, so their elements can be changed using indexing.

Syntax: `ListName[index] = new_value`

Example: `marks = [80, 85, 90]`

`marks[1] = 95`

`print(marks)`

Output

`[80, 95, 90]`

3. Deleting Elements from a List: The `del` statement removes elements from a list.

Syntax

Delete a single element: `del ListName[index]`

Delete multiple elements: `del ListName[start:stop]`

Example 1: `L = [10,20,30,40]`

`del L[2]`

`print(L)`

Output

`[10,20,40]`

2.3.5 Making a True Copy of a List: Using the assignment operator (`=`) does not create a new list. Instead, both variables refer to the same list (Shallow Copy).

Example

`A = [10,20,30]`

`B = A`

`B[0] = 100`

`print(A)`

`print(B)`

Output

`[100,20,30]`

`[100,20,30]`

Note: Both lists change because they refer to the same memory location.

Creating an Independent Copy: Use the `list()` function.

Example

```
A = [10,20,30]
```

```
B = list(A)
```

```
B[0] = 100
```

```
print(A)
```

```
print(B)
```

Output

```
[10,20,30]
```

```
[100,20,30]
```

HINT: RACER PICS + I

**(R->remove, A->append, C->clear, E->extend(), R->reverse, P->pop
I->index, C->count, S->sort)**

2.3.6 List Functions & Methods : Python provides many built-in methods to perform operations on lists.

1. index() Method: The index() method returns the index of the first occurrence of the specified element.

Syntax: ListName.index(item)

Example: L = [13,18,11,16,18]

```
print(L.index(18))
```

Output

2. append() Method: Adds one element at the end of the list.

Syntax: ListName.append(item)

Example: colors = ["Red","Green"]

```
colors.append("Blue")
```

```
print(colors)
```

Output

```
['Red', 'Green', 'Blue']
```

3. extend() Method: The extend() method adds all elements of another list to the end of the current list.

Syntax: ListName.extend(list)

Example: L1 = [1,2,3]

```
L2 = [4,5]
```

```
L1.extend(L2)
```

```
print(L1)
```

Output

```
[1,2,3,4,5]
```

4. insert() Method: The insert() method inserts an element at the specified position.

Syntax: ListName.insert(index,item)

Example: L = ['a','e','u']

```
L.insert(2,'i')
```

```
print(L)
```

Output

```
['a','e','i','u']
```

5. pop() Method: Removes and returns an element from the list.

Syntax: ListName.pop(index)

or

```
ListName.pop()
```

Example

```
L=[10,20,30,40]
```

```
x=L.pop()
```

```
print(x)
```

```
print(L)
```

Output

```
40
```

```
[10,20,30]
```

6. remove() Method: Removes the first occurrence of the specified value.

Syntax: ListName.remove(value)

Example: L=[10,20,30,20]

```
L.remove(20)
```

```
print(L)
```

Output

```
[10,30,20]
```

7. clear() Method: Removes all elements from the list.

Syntax: ListName.clear()

Example: L=[1,2,3]

L.clear()

print(L)

Output

[]

8. count() Method: *Returns the number of times an element appears in the list.*

Syntax

ListName.count(item)

Example

L=[10,20,10,30,10]

print(L.count(10))

Output

3

9. reverse() Method: *Reverses the order of elements in the list.*

Syntax

ListName.reverse()

Example:

L=[1,2,3,4]

L.reverse()

print(L)

Output

[4,3,2,1]

10. sort() Method: *Sorts the list in ascending order by default.*

Syntax

ListName.sort()

Example

L=[40,10,30,20]

L.sort()

print(L)

Output

[10,20,30,40]

Descending Order:

```
L.sort(reverse=True)
```

```
print(L)
```

Output

```
[40,30,20,10]
```

2.4 – Tuples in Python :A Tuple is a built-in data type in Python used to store multiple values in a single variable.

- Tuples are enclosed within parentheses ().
- Elements are separated by commas (,).
- A tuple can store same or different data types.
- Tuples are immutable, which means their elements cannot be changed after creation.

Syntax: TupleName = (value1, value2, value3)

Examples

```
T1 = (1, 2, 3)
```

```
T2 = ('a', 'b', 'c')
```

```
T3 = (10, "Python", 3.5)
```

```
T4 = ()
```

Key Features of Tuples

- Ordered collection
- Immutable (cannot be modified)
- Allows duplicate values
- Can store different data types
- Uses parentheses ()

Important Board Point

Tuples are immutable, whereas Lists are mutable.

2.4.1 Creating Tuples

A tuple can be created in different ways.

1. Creating an Empty Tuple

An empty tuple contains no elements.

Method 1: T = ()

Method 2: T = tuple()

Output: ()

2. Creating a Single Element Tuple: A single element tuple must contain a comma (,) after the element.

Incorrect

```
T = (5)
```

```
print(T)
```

Output

5

Python treats it as an integer, not a tuple.

Correct

```
T = (5,)
```

```
print(T)
```

Output

(5,)

3. Creating a Tuple with Multiple Elements

```
T = (10,20,30)
```

```
print(T)
```

Output

(10,20,30)

4. Creating a Tuple from an Existing Sequence

The tuple() function converts another sequence into a tuple.

Syntax

```
tuple(sequence)
```

The sequence can be:

- String
- List
- Another Tuple

Example 1: From String

```
T = tuple("HELLO")
```

```
print(T)
```

Output

('H','E','L','L','O')

Example 2: From List

```
L = ['P','Y','T','H','O','N']
```

```
T = tuple(L)
print(T)
Output
('P','Y','T','H','O','N')
```

5. Creating Tuple from Keyboard Input

Method 1

```
T = tuple(input("Enter tuple elements: "))
```

Input

12345

Output

```
('1','2','3','4','5')
```

Note: Every input is treated as a string.

Method 2

```
T = eval(input("Enter tuple: "))
```

Example

Input

```
(10,20,30)
```

Output

```
(10,20,30)
```

2.4.2 Tuples vs Lists

Tuples and Lists are similar in many ways, but they also have important differences.

A. Similarities between Tuples and Lists

1. *Length: The len() function returns the total number of elements.*

2. *Indexing: Indexing is used to access individual elements.*

- *Positive indexing starts from 0.*
- *Negative indexing starts from -1.*

3. *Slicing: Slicing extracts a part of the tuple.*

Syntax: TupleName[start:stop]

Example: T = (10,20,30,40,50)

```
print(T[1:4])
```

Output: (20,30,40)

4. Membership Operators: Used to check whether an element exists in a tuple.

Operators: in ,not in

5. Concatenation (+): The + operator joins two tuples.

6. Replication (*): The * operator repeats a tuple.

B. Difference between Tuples and Lists

Tuple	List
Immutable	Mutable
Uses parentheses ()	Uses square brackets []
Elements cannot be modified	Elements can be modified
Faster than lists	Slightly slower than tuples

Example

T = (10,20,30)

T[1] = 50

Output

TypeError

Reason: Tuples are immutable, so their elements cannot be changed.

Difference Between List and Tuple

List	Tuple
Mutable	Immutable
Uses []	Uses ()
Elements can be updated	Elements cannot be updated
Uses list methods like append(), extend(), remove()	Does not support these modification methods

2.4.3 Tuple Operations: *Tuple operations are the different operations that can be performed on tuples.*

2.4.3A Traversing a Tuple: *Traversing a tuple means accessing and processing each element of the tuple one by one.*

A for loop is commonly used to traverse a tuple.

*Syntax: for <item> in <tuple> :
 process the item*

Example:

T = ('P', 'U', 'R', 'E')

for a in T:

print(a)

Output:

P

U

R

E

2.4.3B Joining Tuples: *Two tuples can be joined using the + operator.*

The + operator is called the concatenation operator.

Example:

tpl1 = (1, 3, 5)

tpl2 = (6, 7, 8)

print(tpl1 + tpl2)

Output:

(1, 3, 5, 6, 7, 8)

2.4.3C Repeating or Replicating Tuples: *A tuple can be repeated using the * operator.*

Syntax:

*tuple * number*

Example:

tpl = (1, 3, 5)

*print(tpl * 3)*

Output:

(1, 3, 5, 1, 3, 5, 1, 3, 5)

2.4.3D Slicing a Tuple: Slicing means extracting a part of a tuple.

Syntax: `tuple[start : stop]`

The stop index is not included.

Example:

```
tpl = (10, 12, 14, 20, 22, 24, 30, 32, 34)
```

```
print(tpl[3:-3])
```

Output: (20, 22, 24)

Slicing with Step

To select elements at regular intervals, we use a step.

Syntax:

`tuple[start : stop : step]`

Example 1:

```
tpl = (10, 12, 14, 20, 22, 24, 30, 32, 34)
```

```
print(tpl[0:9:2]) # It means Start from index 0 and select every second element.
```

Output:

(10, 14, 22, 30, 34)

2.4.3E Unpacking Tuples

Packing: Creating a tuple by grouping values together is called packing.

Unpacking: Assigning the elements of a tuple to individual variables is called unpacking.

Syntax:

`variable1, variable2, variable3 = tuple`

The number of variables must normally be equal to the number of tuple elements.

Example:

```
t = (1, 2, 'A', 'B')
```

```
w, x, y, z = t
```

```
print(w, "-", x, "-", y, "-", z)
```

Output:

HINT: MIC LMT

2.4.4 Tuple Functions and Methods

The important tuple functions and methods are:

- 1. len()**
- 2. max()**
- 3. min()**
- 4. index()**
- 5. count()**
- 6. tuple()**

1. len() Function: The len() function returns the number of elements in a tuple.

Syntax: len(tuple)

Example:

```
employee = ('John', 10000, 24, 'Sales')  
print(len(employee))
```

Output:

4

2. max() Function: The max() function returns the maximum value from a tuple.

Syntax: max(tuple)

Example:

```
tpl = (10, 12, 14, 20, 22, 24, 30, 32, 34)  
print(max(tpl))
```

Output:

34

3. min() Function: The min() function returns the minimum value from a tuple.

Syntax: min(tuple)

Example:

```
tpl = (10, 12, 14, 20, 22, 24, 30, 32, 34)  
print(min(tpl))
```

Output:

10

4. index() Method: The `index()` method returns the index position of the first occurrence of a specified element.

Syntax: `tuple.index(item)`

Example:

```
t1 = (3, 4, 5, 6.0)
```

```
print(t1.index(5))
```

Output:

2

If the element is not present, Python raises a `ValueError`.

5. count() Method

The `count()` method returns the number of times an element occurs in a tuple.

Syntax:

`tuple.count(item)`

Example:

```
t1 = (2, 4, 2, 5, 7, 4, 8, 9, 9, 11, 7, 2)
```

```
print(t1.count(2))
```

Output:

3

6. tuple() Function: The `tuple()` function is a constructor used to create a tuple from another sequence or iterable.

Syntax: `tuple(sequence)`

Creating an Empty Tuple

```
t = tuple()
```

```
print(t)
```

Output:

()

Creating a Tuple from a String

```
t = tuple("abc")
```

```
print(t)
```

Output:

('a', 'b', 'c')

Creating a Tuple from a List

```
t = tuple([1, 2, 3])
```

```
print(t)
```

Output:

```
(1, 2, 3)
```

2.5 DICTIONARIES IN PYTHON: A dictionary is a collection of key-value pairs.

A dictionary is written using curly brackets {}.

Example:

```
teachers = {  
    "Dimple": "Computer Science",  
    "Karen": "Sociology",  
    "Harpreet": "Mathematics",  
    "Sabah": "Legal Studies"  
}
```

2.5.1 Creating a Dictionary

Syntax:

```
dictionary_name = {  
    key1: value1,  
    key2: value2  
}
```

Example:

```
student = {  
    "name": "Ravi",  
    "age": 15  
}
```

2.5.2 Accessing Elements of a Dictionary: Dictionary values are accessed using their keys.

Syntax: `dictionary[key]`

Example:

```
fruit = {  
    "apple": 50,  
    "banana": 30  
}
```

```
print(fruit["apple"])
```

Output: 50

Accessing Keys and Values

(i)keys(): The keys() method returns all the keys of a dictionary.

(ii)values(): The values() method returns all the values of a dictionary.

```
d = {  
    "Vowel1": "a",  
    "Vowel2": "e",  
    "Vowel3": "i",  
    "Vowel4": "o",  
    "Vowel5": "u"  
}
```

```
print(d.keys())
```

```
print(d.values())
```

Output:

```
dict_keys(['Vowel1', 'Vowel2', 'Vowel3', 'Vowel4', 'Vowel5'])
```

```
dict_values(['a', 'e', 'i', 'o', 'u'])
```

2.5.3 Characteristics of a Dictionary: The important characteristics of dictionaries are:

1. A dictionary stores data in key:value pairs.
2. A dictionary is mutable.
3. Dictionary values are accessed using keys.
4. Every key in a dictionary must be unique.
5. Keys must be of an immutable/hashable type.
6. A dictionary is not a sequence like a string, list or tuple.
7. Values in a dictionary can be of different data types.
8. Values can be changed using assignment.

2.5.4 Dictionary Operations

Traversing a Dictionary: Traversing a dictionary means accessing and processing its elements one by one. A for loop can be used to traverse a dictionary.

Syntax:

for key in dictionary:

process the key

Example:

```
student = {  
    "name": "Ali",  
    "class": 12  
}
```

for key in student:

print(key, ":", student[key])

Output:

name : Ali

class : 12

Adding Elements to a Dictionary: A new key-value pair can be added using assignment.

Syntax:

dictionary[key] = value

Example:

```
Employee = {  
    'name': 'John',  
    'salary': 10000,  
    'age': 24  
}
```

Employee['dept'] = 'Sales'

print(Employee)

Now the dictionary contains the new key dept.

Important Point

- ***If the key does not exist, a new key-value pair is added.***
- ***If the key already exists, its value is updated.***

2.5.4C Updating Existing Elements:The value of an existing key can be changed using assignment.

Syntax:

dictionary[key] = new_value

Example:

```
Employee = {  
    'name': 'John',  
    'salary': 10000,  
    'age': 24  
}  
Employee['salary'] = 20000  
print(Employee)
```

Result:

```
{'name': 'John', 'salary': 20000, 'age': 24}
```

2.5.4D Deleting Elements from a Dictionary: *There are two common ways to delete elements from a dictionary:*

1. *del*
2. *pop()*

1. del Statement: *The del statement removes a key-value pair from a dictionary.*

Syntax:

```
del dictionary[key]
```

Example:

```
employee = {  
    'salary': 10000,  
    'age': 24,  
    'name': 'John'  
}  
del employee['age']  
print(employee)
```

Output:

```
{'salary': 10000, 'name': 'John'}
```

2. pop() Method: *The pop() method removes the key-value pair and returns the value associated with that key.*

Syntax:

```
dictionary.pop(key)
```

Example:

```
employee = {  
    'salary': 10000,  
    'age': 24,  
    'name': 'John'  
}  
x = employee.pop('age')  
print(x)  
print(employee)
```

Output:

```
24  
{'salary': 10000, 'name': 'John'}  
pop() with Default Value
```

We can provide a default value if the key does not exist.

Syntax:

```
dictionary.pop(key, default)
```

Example:

```
student = {  
    'name': 'Ali',  
    'age': 18  
}  
print(student.pop('class', '12'))
```

Output:

```
12
```

2.5.4E Checking for Existence of a Key: *The membership operators in and not in can be used with dictionaries. They check for the existence of keys.*

Syntax:

key in dictionary

key not in dictionary

in Operator Returns True if the key exists. **not in** Operator Returns True if the key does NOT exists.

```
student = {
```

```
'name': 'Ali',  
'age': 18  
}  
print('age' in student)  
print('marks' not in student)  
OUTPUT:  
True  
True
```

2.5.5 Dictionary Functions and Methods

Important dictionary functions and methods are:

- 1. len()**
- 2. clear()**
- 3. get()**
- 4. items()**
- 5. keys()**
- 6. values()**
- 7. update()**

1. len() Function: The len() function returns the number of key-value pairs in a dictionary.

Syntax:

```
len(dictionary)
```

Example:

```
employee = {  
    'name': 'John',  
    'age': 24  
}
```

```
print(len(employee))
```

Output:

2

2. clear() Method: The clear() method removes all key-value pairs from a dictionary.

Syntax:

```
dictionary.clear()
```

Example:

```
Employee = {  
    'name': 'John',  
    'age': 24  
}
```

```
Employee.clear()
```

```
print(Employee)
```

Output:

```
{}
```

3. get() Method: The `get()` method returns the value associated with a given key.

Syntax:

```
dictionary.get(key)
```

Example:

```
employee = {  
    'name': 'John',  
    'dept': 'Sales'  
}  
print(employee.get('dept'))
```

Output:

Sales

4. items() Method: The `items()` method returns all key-value pairs as (key, value) pairs.

Syntax:

```
dictionary.items()
```

Example:

```
employee = {  
    'name': 'John',  
    'age': 24  
}
```

```
for x in employee.items():
```

```
print(x)
```

Output:

```
('name', 'John')
```

```
('age', 24)
```

5. keys() Method: *The keys() method returns all the keys of a dictionary.*

Syntax:

```
dictionary.keys()
```

Example:

```
employee = {  
    'name': 'John',  
    'age': 24  
}
```

```
print(employee.keys())
```

Output:

```
dict_keys(['name', 'age'])
```

6. values() Method: *The values() method returns all the values of a dictionary.*

Syntax:

```
dictionary.values()
```

Example:

```
employee = {  
    'name': 'John',  
    'age': 24  
}
```

```
print(employee.values())
```

Output:

```
dict_values(['John', 24])
```

7. update() Method: *The update() method adds the key-value pairs of one dictionary to another dictionary. If the same key already exists, its value is replaced.*

Syntax:

```
dictionary1.update(dictionary2)
```

Example:

```
employee1 = {
```

```

    'name': 'John',
    'age': 24
}
employee2 = {
    'name': 'Diya',
    'dept': 'Sales'
}

employee1.update(employee2)

```

```
print(employee1)
```

Output:

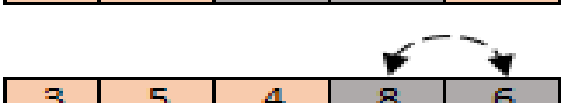
```

{
    'name': 'Diya',
    'age': 24,
    'dept': 'Sales'
}

```

BUBBLE SORT: bubble sort is to compare two adjoining values and exchange them if they are not in proper order.

Bubble sort example

Initial		Initial Unsorted array
Step 1		Compare 1 st and 2 nd (Swap)
Step 2		Compare 2 nd and 3 rd (Do not Swap)
Step 3		Compare 3 rd and 4 th (Swap)
Step 4		Compare 4 th and 5 th (Swap)
Step 5		Repeat Step 1-5 until no more swaps required

PROGRAM ON BUBBLE SORT

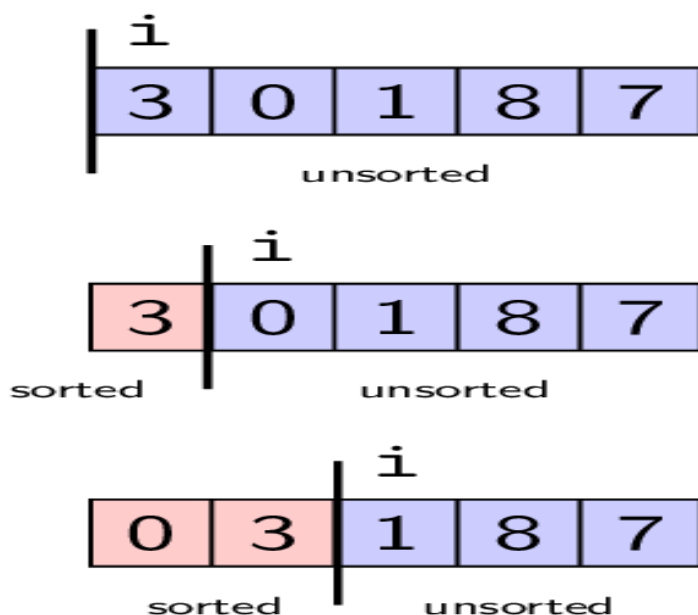
```
a = [5,3,8,4,6]
print("Original list is:", a)
n = len(a)
for i in range(n):
    for j in range(0, n - i - 1):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
print("List after sorting:", a)
```

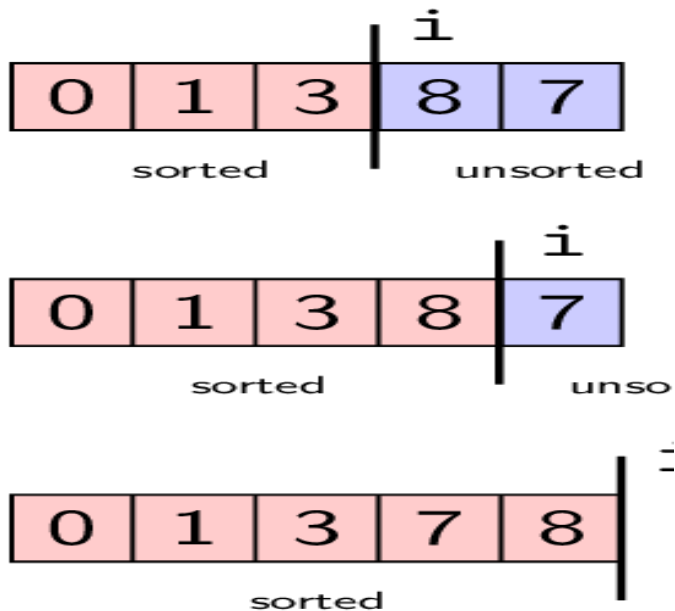
OUTPUT:

Original list is: [5, 3, 8, 4, 6]

List after sorting: [3, 4, 5, 6, 8]

INSERTION SORT: Insertion sort is a sorting algorithm that builds a sorted list one element at a time from the unsorted list by inserting the element at its correct position in sorted list. Below is the example list [3,0,1,8,7]





PROGRAM:

```
a = [3,0,1,8,7]
print("Original list is:", a)
for i in range(1, len(a)):
    key = a[i]
    j = i - 1
    while j >= 0 and key < a[j]:
        a[j + 1] = a[j]
        j = j - 1
    a[j + 1] = key
print("List after sorting:", a)
```

OUTPUT:

Original list is: [3, 0, 1, 8, 7]
 List after sorting: [0, 1, 3, 7, 8]

CHAPTER-III- FUNCTIONS

FUNCTIONS: A Python function is a reusable block of code designed to perform a specific task. Instead of writing the exact same code multiple times, we create it (function) once and use it multiple times.

Python allows three types of functions:

1. built-in functions
2. functions defined in module
3. user defined functions

Types of Functions in Python: There are three main types of functions in Python

1. Built-in Functions

These are functions that are already provided by Python and are always available to use.

Examples: `len()`, `type()`, `int()`, `input()`, etc.

2. Functions in Modules: These are pre-defined functions available inside specific Python modules. To use them, we first need to import the required module.

For **example**, the **`sin()`** function is available in the `math` module, so we need to import `math` before using **`sin()`**.

3. User-Defined Functions: These are functions created by the programmer according to their needs. Programmers can create their own functions and use them in their programs. In this we pass arguments in function call.

Example:

```
def sum(a,b):  
    print(a+b)
```

```
sum(2,3)
```

Output:

```
5
```

Arguments:

There are 2 types of arguments . The arguments provided in the function definition are called **formal argument**, and the arguments provided in the function call are called as **actual arguments**.

Example: `def sum(a,b):` # a, b are formal arguments

```
    c= a+b
```

```
    sum(2,3)      #2,3 are actual arguments
```

Local and global variable:

global variable:

- If a variable is declared outside all the blocks it is called global variable. And the scope of the variable is throughout the program i.e. the global variable can be used anywhere in the program.

Example:

```
def demo():  
    global s          # tell Python that s is the global variable  
    s = "hello abrar" # assign value to the global variable  
  
demo()               # call the function to set s  
print(s)             # print the global variable
```

local variable: If a variable is defined inside any block(if, if-else, loop or function) it is called as local variable. The scope of the variable is the block itself i.e., we can ONLY use local variable inside the block , but NOT outside the block

Example:

```
def sum(a,b):  
    c = a+b # c is local variable because it's inside the function block.  
    print(c)
```

Note : if some variable is provided in both the scopes (local or global), priority will be given to local variable ,but ,if the variable outside the local scope to be accessed inside the block then we use global keyword to make local variable as global variable.

Return keyword : function allows to return a value back to the program using return keyword . Whenever we return a value there should be a variable to hold it or else atleast a print() function to print it to display in the program itself

Example

```
def sum (a,b):  
    return a+b  
  
c= sum(2,4)
```

In the above example,function **sum(2,4)** will return value 6, the variable **c** holds the value 6.

Write a program to find the area of Rectangle using functions.

```
l=int(input("enter length"))  
b=int(input("enter breadth"))  
def Area(l,b):  
    print(l*b)  
Area(l,b) #function call
```

Types of argument

Python supports the following types of arguments

1. positional argument
2. keyword argument
3. default argument

1. Positional Arguments: Arguments that must be passed in the exact order that the function parameters are defined. Below is the example :

```
def sum(a,b):  
    print(a+b)  
sum(7,8)
```

here, in example , 7,8 are positional arguments, value of “a” will become 7 and “b” will become 8 as it follows the proper order.

2. Keyword Arguments: Arguments passed by explicitly naming the parameter, so the order does not matter. Below is the example :

```
def sum(a,b):  
    print(a+b)  
sum(b=7,a=8) #keyword arguments
```

3. Default Arguments : Default argument is a value provided in a function’s definition that is automatically used if you don’t pass a value for that argument in function call. Example:

```
def sum(a=2,b=3):  
    print(a+b)  
sum(10,20) #default value 2,3 is ignored as we are passing 10 & 20  
sum()      #default value 2,3 are taken by a, b respectively as no value is passed here in  
           this function call
```

OUTPUT:

30
5

Write a program which will accept age as an input and print a message whether he/she is eligible for driving or not ?

```
A= int ( input (“enter your age:”) )  
def age (A):  
    if A>=18:  
        print ( “eligible for driving” )
```

```
else :  
    print ( "not eligible for driving ")  
age (A)  
Output:  
enter your age: 23  
eligible
```

Module in Python:

You can create a module by just creating a function in a python file, now this python file say area.py is now our module in which we have some function. We can use the function present inside this "area" module by importing area in another python file.

See the below example program

Write a program to create a module area containing function to calculate square and rectangle areas . Use this functions to calculate the area of square and rectangle by importing in another python program

area.py (user-defined module)

```
def square (s) :  
    return s*s  
def rectangle(l,b):  
    return l*b
```

Using the above module "area" In the below program demo.py

demo.py

```
import area  
square_area = area.square(4)  
rectangle_area = area.rectangle(2,3)
```

Output after executing demo.py

16 6
